PRINCÍPIOS DE ARQUITETURA DE SOFTWARE

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

Sumário

INTRODUÇÃO 4

O QUE É A ARQUITETURA DE SOFTWARE? 5

OBJETIVOS DA ARQUITETURA DE SOFTWARE 6

A FUNÇÃO DO ARQUITETO DE SOFTWARE 7

PRINCÍPIO DA RESPONSABILIDADE ÚNICA 9

PRINCÍPIO ABERTO-FECHADO 10

PRINCÍPIO DA SUBSTITUIÇÃO DE LISKOV 12

PRINCÍPIO DE SEGREGAÇÃO DE INTERFACE 13

PRINCÍPIO DE INVERSÃO DE DEPENDÊNCIA 15

O PRINCÍPIO DA MENOR SURPRESA 16

O PRINCÍPIO DO MENOR ESFORÇO 17

O PRINCÍPIO DO CUSTO DE OPORTUNIDADE 18

O PRINCÍPIO DO ÚLTIMO MOMENTO RESPONSÁVEL 20

PRINCÍPIOS DE DESIGN DE SOFTWARE 21

- SEPARAÇÃO DE INTERESSES 21
- ENCAPSULAMENTO 22
- INVERSÃO DE DEPENDÊNCIA 23

OS PRINCÍPIOS DA BOA ARQUITETURA 25

CONSIDERAÇÕES FINAIS 26

REFERÊNCIAS 27

INTRODUÇÃO

À medida que a complexidade e o tamanho dos sistemas de software têm aumentado, engenheiros de software têm lançado mão de princípios de projeto, tais como a modularização e o ocultamento da informação, de modo a obter sistemas com maior qualidade e a um baixo custo.

Para isso, o projeto da estrutura global do software (arquitetura de software) é uma questão que vem sendo considerada.

Esses pontos de vista compreendem princípios, padrões, antipadrões, regras práticas e essenciais para as tomadas de decisões em direção a uma direção específica e também para avaliar o sucesso do projeto.

Em geral, esses princípios orientarão você para a criação de aplicativos fora de componentes discretos que não têm um acoplamento rígido com outras partes do aplicativo, mas que, em vez disso, se comunicam por meio de interfaces explícitas ou sistemas de mensagens.

O QUE É A ARQUITETURA DE SOFTWARE?

Definição pela IBM

Uma arquitetura é o conjunto de decisões significativas sobre a organização de um sistema de software, a seleção de elementos estruturais e suas interfaces, juntamente com o comportamento especificado nas colaborações entre estes elementos, a composição destes elementos em subsistemas progressivamente maiores e o estilo arquitetural que guia esta organização. (The Rational Unified Process: An Introduction)

Definição pela Microsoft

Arquitetura de Software é o processo de definição de uma solução estruturada que atende a todos os requisitos técnicos e operacionais e ao mesmo tempo otimiza atributos de qualidade padronizados como desempenho, segurança e gerenciamento. Arquitetura envolve uma série de decisões baseadas em uma vasta gama de fatores e cada uma destas decisões pode provocar um impacto considerável no sucesso ou fracasso da aplicação.

Definição pela IEEE

Arquitetura é a organização fundamental de um sistema materializada em seus componentes, na relação entre eles e com o ambiente e nos princípios que guiam seu projeto e evolução.

Em resumo, as definições apresentadas concordam que a Arquitetura de Software tem relação com as tomadas de decisões durante o processo de desenvolvimento assim como a estruturação dos elementos que compõe o sistema. Em poucas palavras:

⟩ Decisões ⟩ Organização fundamental ⟩ Elementos de software ⟩ Relacionamento entre elementos ⟩ Relacionamento com o ambiente ⟩ Princípios que guiam o desenho e evolução ⟩ Pode determinar o sucesso ou fracasso

OBJETIVOS DA ARQUITETURA DE SOFTWARE

Uma série de causas individualmente ou em conjunto pode fracassar um projeto de desenvolvimento de software. Diversos riscos e restrições associados ao processo aumentam a probabilidade de cancelamento. Falta de controle, escopo volátil, prazos exorbitantes, falta de recursos e entre outros vários motivos.

Shaw e Garlan (96) a arquitetura define o que é o sistema em termos de componentes computacionais e, os relacionamentos entre estes componentes, os padrões que guiam a sua composição e restrições.

Além da escolha dos algoritmos e estruturas de dados, a arquitetura envolve: decisões sobre as estruturas que formarão o sistema, controle, protocolos de comunicação, sincronização e acesso a dados, atribuição de funcionalidade a elementos do sistema, distribuição física dos elementos escalabilidade e desempenho e outros atributos de qualidade; e seleção de alternativas de projeto.

A FUNÇÃO DO ARQUITETO DE SOFTWARE

O profissional Arquiteto de Software é responsável por uma grande variedade de atividades dentro do processo de desenvolvimento de sistemas. Como é ele quem define a arquitetura do software que será construído, ele tem a missão de garantir que toda a equipe seguirá as diretrizes da arquitetura até o final do projeto.

Além disso, faz parte do trabalho de um Arquiteto de Software envolver-se com todo o processo de desenvolvimento, comunicar a arquitetura os interessados pelo projeto, apoiar desenvolvedores na implementação e por ser um líder técnico, também terá a função de mentor e ajudará na formação de novos arquitetos.

- Definir, criar e manter a documentação arquitetural para guiar a construção e manutenção do sistema.
- Definir estratégias, estilos e padrões arquiteturais.
- Garantir que as diretrizes da arquitetura serão aplicadas até o final do projeto.
- Envolver-se com todo o processo de desenvolvimento.
- Comunicar a arquitetura aos stakeholders.
- Apoiar desenvolvedores na implementação.
- Atuar como mentor e formar novos arquitetos.

Durante todo o projeto de desenvolvimento de um software, o Arquiteto será cobrado, ou melhor, será exigido de forma a garantir certas necessidades do projeto e atender a determinadas restrições que são impostas do início ao fim. O Arquiteto será cobrado pelo desenvolvimento ágil, com qualidade e de baixo custo. O atendimento de requisitos funcionais e não-funcionais severos, além de sua própria experiência, do seu time, do contexto organizacional e ambiente técnico que podem influenciar positivamente ou negativamente.

Uma das habilidades vitais de um arquiteto é poder visualizar a arquitetura de software de vários pontos de vista diferentes: cada uma delas individualmente pode não ser totalmente relevante, mas combiná-las oferece uma visão de alto nível do produto.

Arquitetos de soluções são os especialistas designados responsáveis pela arquitetura de software de um sistema, bem como os padrões técnicos (incluindo tecnologias, plataformas, infraestrutura) de um produto específico. Eles definem a visão e sua análise é essencial para a definição, o design, a entrega e o suporte ao longo da vida útil do produto. Portanto, eles precisam entender não apenas o que os negócios precisam, mas também o que é lógico, escalável, econômico e alinhado com os objetivos gerais de tecnologia da organização.

PRINCÍPIO DA RESPONSABILIDADE ÚNICA

Cada capacidade do sistema (por exemplo, serviço/módulo/API) deve ter apenas uma responsabilidade e um motivo para mudar. Manter as responsabilidades o mais restritas possível significa que os usuários sabem da finalidade pretendida, o que leva a menos erros.

A reposta é simples: quanto maior o número de responsabilidades, maiores as chances de modificação dessa classe no futuro e maiores as chances de inserção de bugs que atrapalharão a classe por inteiro.

A extensibilidade! É comum a manutenção de códigos. Você deve, então, lidar com as classes já criadas e, possivelmente, adicionar ou remover funcionalidades. Para uma classe 'não-extensível', a adição ou remoção de funcionalidades implica na modificação da classe por si só. Ao fazer tal mudança, você estará propenso a fazer com que outras partes do seu código não funcionem mais.

PRINCÍPIO ABERTO-FECHADO

Este princípio postula que é preferível estender um comportamento do sistema, sem modificá-lo. Embora muitas vezes não seja uma boa ideia tentar antecipar alterações nos requisitos com antecedência (pois isso pode levar a projetos excessivamente complexos), poder adaptar novas funcionalidades com alterações mínimas nos componentes existentes é essencial para a longevidade do aplicativo.

O princípio de Aberto/Fechado propõe que entidades (classes, funções, módulos, etc.) devem ser abertas para extensão, mas fechadas para modificação.

Aberto para extensão significa que, ao receber um novo requerimento, é possível adicionar um novo comportamento. Fechado para modificação significa que, para introduzir um novo comportamento (extensão), não é necessário modificar o código existente.

Dependency Injection nada mais é do que passar uma dependência como argumento para uma entidade qualquer. Isso permite que o código dependa mais de uma abstração do que de algo concreto. No exemplo de código acima, a classe Mailer pode receber um template qualquer, mas por padrão ela recebe WelcomeTemplate.

Dependency Injection é apenas uma maneira de estender uma classe sem alterar seu comportamento. Existem muitas outras soluções como Decorator Pattern, Strategy Pattern, Composição e Herança.

Entre essas soluções, é importante atentar-se a Herança. Apesar da herança permitir resolver o OCP, se não houver o devido cuidado, ela pode aumentar o acoplamento do código, dificultando a manutenção, extensão e permitindo criar código que viole o Single Responsibility Principle.

O problema é que a herança pode fazer com que aumente o acoplamento entre as classes, se dependermos mais da forma como a classe é implementada, do que da maneira como ela é representada no sistema (sua abstração).

O ideal é que a entidade dependa de uma abstração que forneça uma interface para interagirmos, sem nos importarmos como ela funciona.

PRINCÍPIO DA SUBSTITUIÇÃO DE LISKOV

O princípio da Substituição de Liskov leva o nome da sua criadora Barbara Liskov, que introduziu o conceito deste princípio em uma conferência em 1987 e posteriormente, em 1994 no artigo *Family Values: A Behavioral Notion of Subtyping* com a parceria de Jeannette Wing. Podemos dizer que sua definição original de forma resumida é a seguinte:

Seja q(x) uma propriedade que se pode provar do objeto x do tipo T. Então, q(y) também é possível provar para o objeto y do tipo S, sendo S um subtipo de T.

O que isso quer dizer afinal?

Significa dizer que classes derivadas devem poder substituídas por suas classes base e que classes base pode ser substituídas por qualquer uma das suas subclasses. Uma subclasse deve sobrescrever os métodos da superclasse de forma que a funcionalidade do ponto de vista do cliente continue a mesma.

No desenvolvimento de software, isso significa que as classes derivadas devem ser substituídas por suas classes base, mas a semelhança desse princípio com o Design by Contract de Bertrand Meyer é como ela pode ser aplicada à arquitetura de software distribuída: dois serviços se comunicam de forma eficaz e repetida quando há um contrato comum ‘entre eles, que define as entradas/saídas, sua estrutura e suas restrições. Portanto: dados dois componentes distribuídos com o mesmo contrato, um deve ser substituível por outro componente com o mesmo contrato, sem alterar a correção do sistema.

PRINCÍPIO DE SEGREGAÇÃO DE INTERFACE

As interfaces/contratos devem ser o mais detalhados possível e específicos do cliente, para que os clientes que se ligam não dependam da funcionalidade que não usam. Isso anda de mãos dadas com o princípio da responsabilidade única: ao quebrar as interfaces, favorecemos a composição separando por papéis/responsabilidades e o desacoplamento por não acoplar módulos derivativos a responsabilidades desnecessárias.

Em programação orientada a objetos, quando falamos de interface, estamos falando do conjunto de métodos que um objeto expõe, ou seja, das maneiras como nós podemos interagir com esse objeto. Toda mensagem (ou chamada de método) que um objeto recebe constitui uma interface.

A interface funciona como um contrato: nós definimos o comportamento da interface na forma de diferentes métodos que ela possui. Cada classe que deseja compartilhar o comportamento dessa interface precisa implementar os métodos dela. Quando a classe utiliza uma interface, ela assina esse contrato dizendo que irá implementar todos os métodos dessa interface.

As interfaces são comumente utilizadas por linguagens de programação estaticamente tipadas (como Java e C#) para adicionar novos comportamentos às classes, isso é feito utilizando a palavra reservada interface.

Linguagens como Ruby ou Python, definem sua interface de forma implícita, através dos métodos que a classe implementa. Comumente nos referimos a essa propriedade como *Duck Typing*. A ideia por trás do *Duck Typing* é que o objeto é representado pelos métodos que ele possui e pelo que esses métodos fazem.

Outra forma de implementar interfaces em Ruby é utilizando herança. Nós podemos fazer com que a classe base lance exceções para cada método, de forma que a classe herdeira é obrigada a implementar esses métodos. O mesmo pode ser obtido com mixins. Dessa maneira, ao herdar uma classe ou incluir um módulo, a classe herdeira precisa implementar o método.

O ISP é violado quando uma classe é obrigada a implementar métodos que não utiliza. Por esse motivo, o ISP diz que as interfaces devem ser específicas (ou pequenas) para que as classes possam implementar somente os comportamentos necessários. Caso contrário, erros serão lançados.

PRINCÍPIO DE INVERSÃO DE DEPENDÊNCIA

Módulos de alto nível não devem depender dos de baixo nível; ambos devem depender de abstrações. Da mesma forma, as abstrações não devem depender de detalhes, mas os detalhes devem depender de abstrações. Como tal, esse princípio introduz uma abstração de interface entre componentes ou camadas de software de nível superior e inferior para remover as dependências entre eles.

O princípio da inversão de dependência refere-se à dissociação de módulos de software. Dessa forma, em vez de módulos de alto nível, dependendo de módulos de baixo nível, ambos dependerão de abstrações.

Para cumprir esse princípio, precisamos usar um padrão de design conhecido como padrão de inversão de dependência , geralmente resolvido usando injeção de dependência .

A injeção de dependência é um tópico enorme e pode ser tão complicado ou simples quanto se possa perceber.

Normalmente, a injeção de dependência é usada simplesmente ‘injetando’ quaisquer dependências de uma classe através do construtor da classe ‘como um parâmetro de entrada’.

O PRINCÍPIO DA MENOR SURPRESA

O princípio de menos espanto (ou menos surpresa) sugere que uma solução ou abordagem não surpreenderia uma pessoa razoavelmente experiente na área de assunto quando encontrada pela primeira vez (o público pode variar, por exemplo, usuário final, programador, testador etc.).

Em termos mais práticos, o princípio visa alavancar o conhecimento pré-existente dos usuários para minimizar sua curva de aprendizado ao usar um módulo; portanto, qualquer coisa com alto fator de imprevisibilidade é um bom candidato para o redesenho.

Aplica-se a todos os aspectos da arquitetura de software: dos serviços de nomeação, à visualização de interfaces do usuário e ao design do modelo de domínio.

O PRINCÍPIO DO MENOR ESFORÇO

Seu princípio (também chamado de Lei de Zipf) deriva de um comportamento humano básico: todo mundo tende a seguir o caminho que é o mais próximo possível do esforço. Por exemplo, se nosso design seguir um padrão específico, o próximo desenvolvedor seguirá o mesmo padrão repetidamente, a menos que haja uma maneira significativamente mais fácil de executar a tarefa; nesse caso, eles serão alterados! Ou, levando isso adiante, uma vez que eles encontram resultados aceitáveis para uma tarefa, não há necessidade imediata de melhorar a solução atual.

Como tal, é imperativo almejar um começo forte, colocando a arquitetura de software certa no lugar: estabelece altas expectativas e garante que todos entendam que a qualidade não é comprometida no ciclo de vida do projeto e será respeitada em caso de mudanças futuras.

A grandeza desse princípio reside no fato de que seus benefícios extrapolam: uma vez que implementamos um design correto, podemos criar uma estrutura arquitetônica que será a base dos próximos sistemas que construímos. Em outras palavras, somos capazes de estabelecer um modelo bem-sucedido e à prova de futuro para os sistemas de software da organização.

O PRINCÍPIO DO CUSTO DE OPORTUNIDADE

Toda vez que fazemos uma escolha, há um certo valor que colocamos nessa escolha. O valor tem duas partes: benefícios e custos. O custo de oportunidade de uma escolha é o que desistimos de obtê-la. Para tomar uma boa decisão econômica, queremos escolher a opção com o maior benefício para nós, mas com o menor custo. Por exemplo, se tivermos duas opções, um sistema interno ou um produto de fornecedor pronto para uso e escolhermos o último, nosso custo de oportunidade será o novo sistema brilhante que nossa equipe de desenvolvimento poderia ter desenvolvido, mas não o fez . É disso que se trata a arquitetura de software: ponderar as opções umas das outras e tentar tomar uma decisão informada sobre qual delas agregará mais valor ao projeto. Por exemplo, uma dicotomia muito comum é criar uma solução tática com tempo de comercialização rápido ou uma solução mais estratégica, que agora será mais cara, com o objetivo de alavancá-la em projetos futuros e, portanto, minimizar o custo posteriormente. Aqui estão alguns pontos a considerar:

- Qual é o tempo disponível para a análise ou avaliação arquitetônica?
- Qual é o pipeline de produtos para os próximos 1 a 3 anos? E que outros projetos estão alinhados? Você consegue ver alguma sinergia?
- Qual é a sua dívida técnica atual que você poderia enfrentar?
- E, invertendo isso: quanta dívida técnica nova incorrerá se você buscar uma solução tática?
- Quais atributos de qualidade tendem a ser os mais importantes para os sistemas em sua organização e como eles serão comprometidos pela solução proposta?
- Além da equipe de arquitetura de software, quem mais é uma parte interessada que afetará a decisão? O negócio? Seu chefe?
- A Autoridade de Projeto Técnico?

- Quais são os principais objetivos de cada parte interessada?
- Como você irá mitigar necessidades conflitantes?

O PRINCÍPIO DO ÚLTIMO MOMENTO RESPONSÁVEL

Esse princípio (também conhecido como Custo do atraso) é originado do Lean Software Development e enfatiza a realização de ações importantes e decisões cruciais pelo maior tempo possível. Isso é feito para não eliminar alternativas importantes até o último momento possível, ou seja, aguarde para restringir as opções até que você esteja melhor informado.

Uma estratégia de não tomar uma decisão prematura, mas adiar o compromisso e manter em aberto decisões importantes e irreversíveis até que o custo de não tomar uma decisão se torne maior que o custo de tomar uma decisão.

Uma maneira de mitigar o risco de decidir tarde demais é criar a Prova de Conceitos (POCs) para prototipar as opções alternativas e demonstrar às partes interessadas o que elas estão pedindo.

Os princípios de arquitetura de software nos ajudam a avaliar as decisões que tomamos ao longo do projeto e também para garantir que estamos alinhados com os objetivos gerais, não apenas para o projeto, mas também a tecnologia da organização.

SEPARAÇÃO DE INTERESSES

Um princípio norteador durante o desenvolvimento é o da Separação de Interesses. Esse princípio declara que o software deve ser separado de acordo com os tipos de trabalho que ele executa. Por exemplo, considere um aplicativo que inclua uma lógica para identificar itens importantes a serem exibidos ao usuário, e que formata esses itens de uma maneira específica para torná-los mais perceptíveis.

O comportamento responsável por escolher os itens a serem formatados deve ser mantido separado do comportamento responsável por formatar os itens, uma vez que esses comportamentos são questões separadas que são relacionadas apenas coincidentes entre si.

Em termos de arquitetura, os aplicativos podem ser logicamente criados para seguir esse princípio, separando o comportamento principal de negócios da infraestrutura e da lógica da interface do usuário.

O ideal é que a lógica e as regras de negócio residam em um projeto separado, que não deve depender de outros projetos no aplicativo.

Essa separação ajuda a garantir que o modelo de negócios seja fácil de testar e possa evoluir sem estar rigidamente acoplado a detalhes de implementação de baixo nível.

A separação de interesses é uma consideração fundamental por trás do uso de camadas em arquiteturas de aplicativo.

ENCAPSULAMENTO

Diferentes partes de um aplicativo devem usar o encapsulamento para isolá-las de outras partes do aplicativo. As camadas e os componentes do aplicativo devem poder ajustar sua implementação interna sem dividir seus colaboradores, desde que contratos externos não sejam violados.

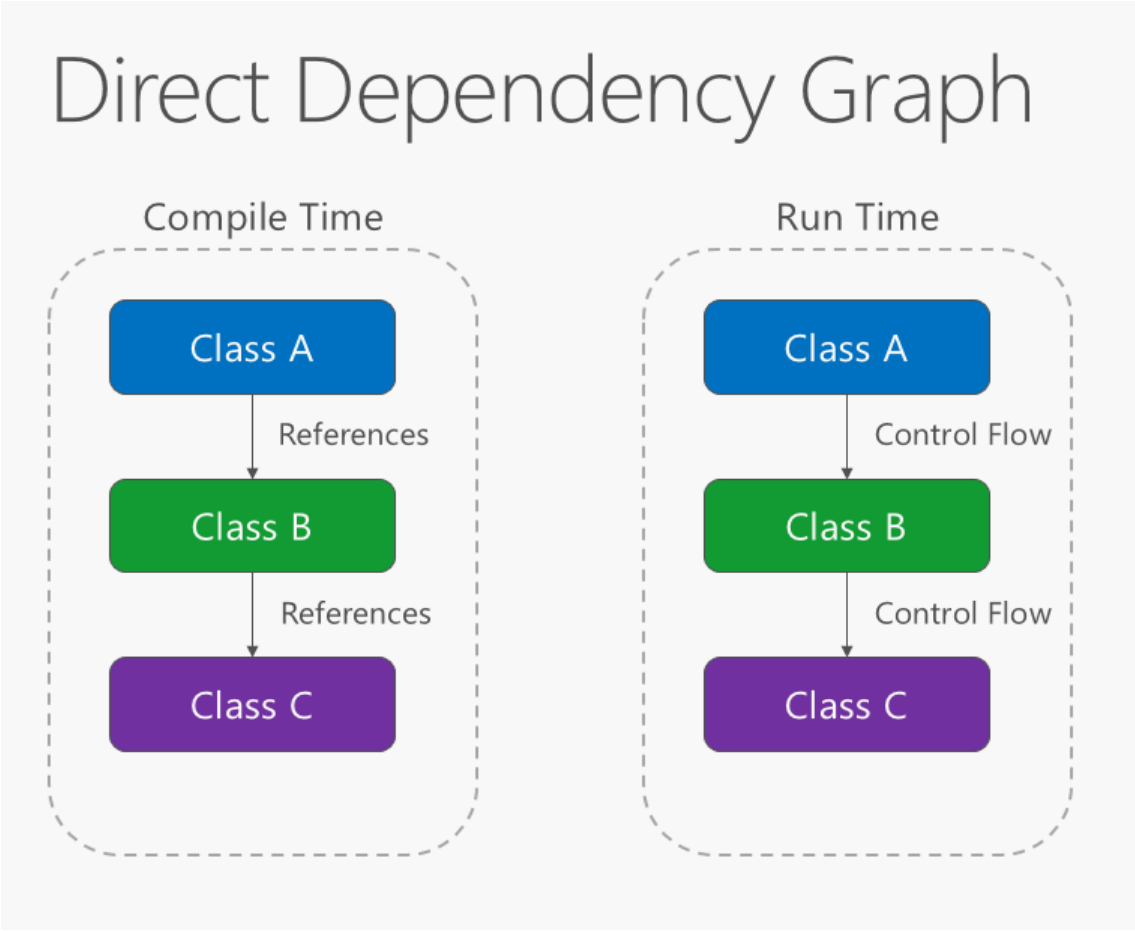
O uso adequado do encapsulamento ajuda a obter um acoplamento flexível e uma modularidade nos designs do aplicativo, pois os objetos e os pacotes podem ser substituídos por implementações alternativas, desde que a mesma interface seja mantida.

Nas classes, o encapsulamento é obtido por meio da limitação do acesso externo ao estado interno da classe. Se um ator externo desejar manipular o estado do objeto, ele deverá fazer isso por meio de uma função bem definida (ou um setter de propriedade), em vez de ter acesso direto ao estado particular do objeto.

Da mesma forma, os componentes do aplicativo e os próprios aplicativos devem expor interfaces bem definidas para uso de seus colaboradores, em vez de permitir que seu estado seja modificado diretamente.

Essa abordagem libera o design interno do aplicativo para evoluir ao longo do tempo sem se preocupar que isso fará com que os colaboradores quebrem, desde que os contratos públicos sejam mantidos.

A direção da dependência dentro do aplicativo deve ser na direção de abstração, não os detalhes de implementação. A maioria dos aplicativos é escrita de forma que os fluxos de dependência de tempo de compilação estejam na direção da execução do tempo de execução, produzindo um grafo de dependência direta. Ou seja, se A classe A chamar um método da classe B e a classe B chamar um método da classe C, em seguida, em uma classe de tempo de compilação A dependerá da classe B, e a classe B dependerá da classe C, como mostra a Figura abaixo:



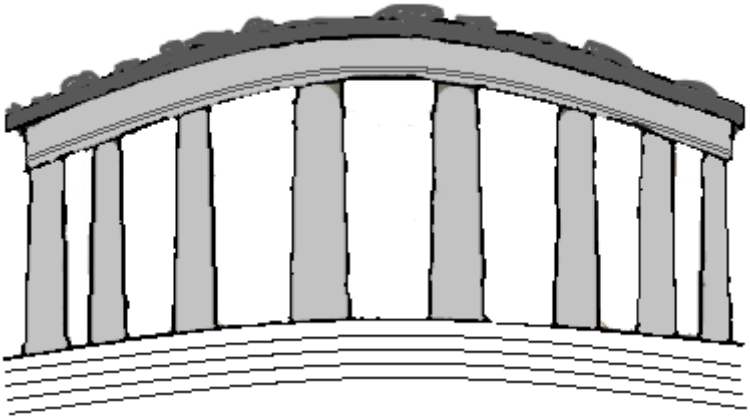
(Fonte: https://docs.microsoft.com/pt-br/dotnet/architecture/modern-web-apps-azure/architectural-principles)

A aplicação do princípio da inversão de dependência permite que A chame métodos em uma abstração implementada por B, possibilitando que A chame B em runtime, mas que B dependa de uma interface controlada por A em tempo de compilação (*invertendo*, portanto, a dependência típica em tempo de compilação). Em tempo de execução, o fluxo da execução do programa permanece inalterado, mas a introdução de interfaces significa que diferentes implementações dessas interfaces podem ser conectadas com facilidade.

A inversão de dependência é uma parte fundamental da criação de aplicativos menos rígidos, já que os detalhes da implementação podem ser escritos para depender e implementar abstrações de nível superior, em vez do contrário. Como resultado, os aplicativos resultantes são mais testáveis, modulares e de manutenção mais fácil. A prática da injeção de dependência se tornou possível pela observância do princípio da inversão de dependência.

OS PRINCÍPIOS DA BOA ARQUITETURA

Já nos tempos antigos, os princípios de uma arquitetura bem projetada eram conhecidos e descritos, por exemplo, pelo arquiteto romano Vitruvius em 'De architectura'; " Firmitas, utilitas, venustas". A arquitetura deve ser estável, útil e bonita.



(https://oieduardorabelo.medium.com/flexibilidade-um-princ%C3%ADpio-de-arquitetura-de-software-de1d9e2314bb)

Para a arquitetura de software, os princípios de estabilidade e utilidade representam a resiliência e o ciclo de vida do sistema, e estão intimamente relacionados a como a arquitetura resiste que permite mudanças ao longo do tempo. O princípio da beleza surge quando a arquitetura está em harmonia com o ambiente e sem desordem e complexidade desnecessárias.

Uma arquitetura de software flexível é capaz de se adaptar às mudanças nos requisitos de ambiente e usabilidade, sem abranger mudanças estruturais. Também é livre de estruturas rígidas que de outro modo obstruiriam a evolução e o crescimento funcional.

CONSIDERAÇÕES FINAIS

Certamente há cenários em que o foco inicial na flexibilidade da arquitetura é de menor importância. No entanto, lembre-se de que até mesmo um aplicativo independente que nunca deveria ser alterado pode, de repente, ser obrigado a integrar-se a sistemas externos ou evoluir com novas funcionalidades.

Para a maioria dos sistemas críticos em larga escala e de negócios, é necessário investir esforços no projeto de uma boa arquitetura para garantir que eles possam possibilitar mudanças, alcançar a longevidade e ter uma boa aparência em todos os estágios de seus ciclos de vida.

REFERÊNCIAS

BENJAMIM, André. **Princípio Aberto/Fechado. 2**020.

CASTRO, Victor. **Responsabilidade única e aberto/fechado**. DTI Digital. 2018.

FELIPE, Marcos. **O Princípio da Substituição de Liskov**. 2018.

HENRIQUE, Jefferson. **5 princípios fundamentais da arquitetura de software**. Pastelaria de software. 2020.

LOPES, Felipe Tório. **Princípios e Práticas em Arquitetura de Software.** Instituto de Gestão em Tecnologia da Informação.2016.

NAKAGAWA, Elisa Yumi. **Análise e Projeto Orientados a Objeto,** SCE 526. 2013.

RABELO, Eduardo**. Flexibilidade: Um princípio de Arquitetura de Software.** 2019.

ROBERTO, Jones. **Princípio de inversão de dependência.** 2019.

VERGÍLIO, Silvia Regina. **Arquitetura de Software.**